Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

MiniGit: A Lightweight Version Control System Using C++ and Data Structures

Team Name: *Team #*	*Blaze* *DSCPP-III-2026-T296*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Gangwar, Abhinesh – 2510370192* *2510370192@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Jain, Sparsh – 2510370159* *2510370159@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Bajaj, Krish – 2510370469* *2510370469@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Joshi, Snehil – 2510370146* *2510370146@geu.ac.in* 

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

MiniGit is a lightweight, local, command-line version-control system written in C++. It explains the core idea of version control in a simple form: create a repository, track files, save versions with messages, view history, and restore earlier work, using classic data structures.

In Phase 3 the team completed the whole proposal. A user can now restore any saved commit with `checkout`, reverse the last restore with `rollback`, create and switch branches, merge a branch using the nearest common ancestor, and draw the history with `log --graph`. Storage was improved with tree objects and packfile compression, and the init and CLI test gaps from Phase 2 were closed.

The product is one standalone executable with no external library, plus a sample repository, a README and user guide, and a documentation site. All four member modules work together through one command dispatcher.

Updated Project Approach and Architecture

MiniGit keeps the 5-layer modular design from Phase 2: (1) CLI interface, (2) command handler, (3) repository and staging manager, (4) ignore engine, (5) storage layer. Every module returns a plain result object (`AddResult`, `CommitResult`, `CheckoutResult`, `MergeResult`) and never prints; only the CLI prints, so each module is easy to test.

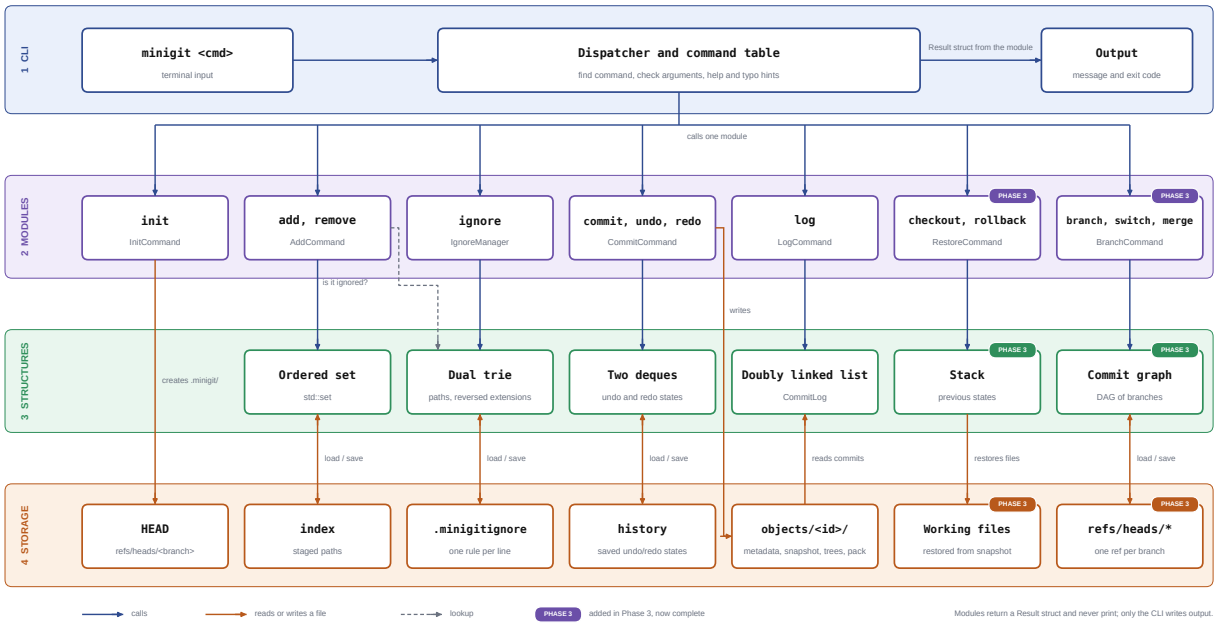
New in Phase 3: the repository layer gained a commit graph and branch references (`refs/heads/<name>`); the storage layer gained tree objects and a packfile.

Data structures:

- **Dual Trie:** forward Trie plus reversed-extension Trie for ignore rules; O(L) check. Also used so restore never overwrites ignored files.
- **Set** (`std::set`): staging index, no duplicates.
- **Deque:** undo/redo states, history limit 20.
- **Doubly linked list:** `log` in both directions.
- **Stack:** previous states for `rollback`.
- **Commit graph (DAG):** each commit stores its parents; a queue and a visited set find the common ancestor for merge and print `log --graph`.
- **Map:** branch name → commit ID.

Storage and libraries: `.minigit` holds HEAD, index, history, branch refs and `objects/`. Commit IDs are 40-character SHA-1 hashes from a self-contained implementation. Only the C++ standard library is used, with a bundled filesystem fallback for C++14/C++17 compilers. No network protocol is used; MiniGit is fully local.

Updated Architecture Diagram



Tasks Completed

Task Completed	Team Member
Dispatcher entries and help text for `checkout`, `rollback`, `branch`, `switch` and `merge`; result-to-message mapping for the new commands; tests for `init` and the CLI.	Abhinesh Gangwar
Sample repository and demo walkthrough; README user guide with command examples; ignore rules applied during restore, switch and merge; extra Dual Trie and staging tests for these flows.	Sparsh Jain
`rollback` with a stack of recent states, kept consistent with HEAD, index and undo/redo; tree objects; packfile compression with a round-trip check; `merge <branch>` using the nearest common ancestor with conflict markers; rollback, storage and merge tests.	Krish Bajaj
`checkout <commit-id>` restores the files of a saved commit from its snapshot; changes in the working folder are checked first and ignored files are never touched. `branch`, `switch` and branch references; commit graph (DAG) with parent links; `log --graph`. Checkout tests, branch tests.	Snehil Joshi
Phase 2 work (`init`, `add`, `remove`, `ignore`, `commit`, `undo`, `redo`, `log`) is unchanged and its tests still pass.	All members

Challenges/Roadblocks

- **Preventing accidental data loss during checkout:** Added checks for staged and modified files before switching commits to avoid losing unsaved work.
- **Keeping ignored files safe:** Used the Dual Trie to check file paths before restore, checkout and merge operations.
- **Maintaining repository consistency:** Updated HEAD, the staging index and undo/redo history together to prevent inconsistent states.
- **Handling branches with multiple parents:** Used BFS from both branches to find their nearest common ancestor, including merge commits.
- **Dealing with conflicting file changes:** Added conflict markers for files changed in both branches and prevented partial updates when a merge fails.
- **Avoiding data corruption during compression:** Verified packed objects before deleting their original copies, while keeping older objects accessible.
- **Making the project work across Windows environments:** Used `normalizePath()` and a filesystem fallback to handle path and compiler differences.
- **Defining the project's scope:** Kept MiniGit focused on local text-file version control, without remote repository support.

Project Outcome/Deliverables

MiniGit is a working terminal-based version-control application in C++ for local text files. The core operations are `init`, `add`, `remove`, `ignore`, `commit`, `undo`, `redo`, `log`, `checkout` and `rollback`; the extensions are `branch`, `switch`, `merge` and `log --graph`.

The submission includes organized C++ source code, unit and manual tests, a sample repository with a demo walkthrough, a README and user guide with command examples, a documentation site, this report, and a live demonstration.

The application builds into one standalone executable with no external dependency, and a first-run `install` step lets the user type `minigit` from any terminal.

Progress Overview

All six core operations in the proposal (`init`, `add`, `commit`, `log`, `checkout`, `rollback`) are complete and tested, so the core scope is 100% done. All four member modules are finished and joined through one command dispatcher.

Ahead of schedule: the Trie-based ignore engine, deque undo/redo, `remove`, first-run installation, branching with the commit graph, `merge`, `log --graph`, tree objects and packfile compression.

Behind schedule: nothing. The Phase 2 gaps (checkout, rollback, init and CLI tests, sample repository) are closed.

Still open: final polish for the live demonstration. Remote repositories and rebase are future scope only.

Codebase Information

Repository Link: `https://github.com/strange-exe/MiniGit`
Documentation Site: `https://abhinesh.codes/MiniGit/`
Active Branch: `main` holds the integrated code. Each member works in an individual branch and merges into `main` through a pull request after tests pass.
Layout: `module/include`, `module/src`, `module/tests`, `Reports/`, `sample_repo/`, `README.md`, `DOCUMENTATION.md`

Important Commits:

- Repository setup, CLI parser and dispatcher; `install` (Abhinesh)
- Trie, Ignore Manager, Staging Area, `add`/`remove`/`ignore`, tests (Sparsh)
- SHA-1 commit IDs, `undo`/`redo`; `rollback`, tree objects, packfile, `merge` (Krish)
- `CommitLog`, `log`; `checkout`, branches, commit graph, `log --graph` (Snehil)
- Sample repository, README and documentation site merged into `main`

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Unit test: Trie (test_trie)	Pass	Checked insert, search, deletion, pruning and path matching.
Unit test: Add and Ignore (test_add)	Pass	Tested file staging, duplicate handling, ignore rules and index saving.
Unit test: Commit (test_commit)	Pass	All 18 checks passed, covering commit IDs, metadata, parent links and undo/redo.
Unit test: Log (test_commitlog, test_log)	Pass	Tested log searching, navigation, sorting and empty repository handling.
Unit test: Checkout and Rollback	Pass	Verified file restoration, unsaved change protection and ignored file handling.
Unit test: Branch, Graph, Merge	Pass	Tested ancestor detection, clean merges and conflict handling.
Unit test: Tree objects and Packfile	Pass	Verified packed object integrity and access to older objects.
Unit test: Init and CLI	Pass	Checked repository setup, repeated initialization and invalid commands.
Manual run on sample repository	Pass	Tested the complete workflow, including commits, branches, merges, checkout and rollback.

Deliverables Progress

- **Core CLI pipeline** (`init`, `add`, `commit`, `log`, `checkout`, `rollback`): Completed
- **Dual Trie ignore engine** (`.minigitignore`): Completed
- **Deque undo/redo integration:** Completed
- **Branching, commit graph and merge** (`branch`, `switch`, `merge`, `log --graph`): Completed
- **Tree objects and packfile compression:** Completed
- **Sample repository, README and documentation site:** Completed
- **Final report and live demonstration:** Report completed; demo ready for evaluation